SENIOR / SDET LEVEL

10 Real Playwright Interview Questions

That nobody talks about

Not definitions. Not theory.
Experience-based questions from real startup QA interviews.

10	5	7+
Questions	Categories	Years Experience

Curated by Monica CV Rao
Senior QA Automation Engineer | Founding QA Lead | SDET
linkedin.com/in/monica-cv

FRAMEWORK & ARCHITECTURE

Q1 How would you design a Playwright framework from scratch for a startup with 3 developers and no existing tests?

Real context: You're the first QA hire. Day 1. Nothing exists.

What a strong answer looks like:

Don't over-engineer it on day one. Start lean — one test runner, one reporter, one CI hook. A strong answer shows prioritisation under constraints:

- Start with smoke tests for the 3 most critical user journeys
- Use Page Object Model from day one so it scales without rewrites
- Integrate into GitHub Actions immediately — even if it's just 5 tests
- Document your decisions so the team understands the 'why'
- Plan for growth: folder structure that supports 500 tests, not just 5

What to add: Mention that in startups, speed of setup matters as much as quality of setup. A framework nobody uses is worthless.

Q2 How do you structure your Playwright project when the product is changing fast and tests keep breaking?

Real context: Startup is pivoting. UI changes weekly. Your tests are a maintenance nightmare.

What a strong answer looks like:

Stability comes from selector strategy and test isolation. Strong candidates talk about:

- Using data-testid attributes agreed with devs — never CSS or XPath
- Keeping tests independent — no shared state between tests
- Separating test data from test logic
- Having a 'contract' with devs: if you add data-testid, I own the test
- Running only smoke tests on feature branches, full regression on main

What to add: Mention the importance of QA being part of the dev process, not downstream of it.

FLAKY TESTS & STABILITY

Q3 Your Playwright tests pass locally but fail consistently in CI. Walk me through exactly how you debug this.

Real context: This happens to every QA engineer. How you handle it reveals your depth.

What a strong answer looks like:

A senior engineer has a systematic approach, not guesswork:

- Check for environment differences first: Node version, browser version, OS
- Run tests in headed mode locally with same CI environment variables
- Look at Playwright traces and screenshots — trace.zip is your best friend
- Check for timing issues: network speed in CI vs local
- Look for hardcoded waits vs proper waitForSelector or waitForLoadState
- Check if tests share state — database, cookies, localStorage

What to add: Show you use Playwright's built-in debugging tools — traces, video, screenshots on failure.

Q4 How do you handle authentication across 50+ test cases without logging in every single time?

Real context: Every startup has auth. How you handle it separates juniors from seniors.

What a strong answer looks like:

This is a classic Playwright architecture question. Strong answer covers storageState:

- Use Playwright's storageState to save auth cookies/tokens once
- Create a global setup file that logs in once and saves state to a file
- Reference that state file in your playwright.config.ts
- Use different state files for different user roles (admin, viewer, editor)
- Re-generate state file in CI as part of setup, not per test

What to add: Mention that this cuts test suite runtime dramatically in startups where every CI minute costs money.

CI/CD INTEGRATION

Q5 How do you decide which Playwright tests run on every PR vs only on merge to main?

Real context: CI is slow. Devs are complaining. You need a strategy.

What a strong answer looks like:

This is a risk-based testing decision. Strong candidates show strategic thinking:

- PR: smoke tests only — critical paths, login, core user journey (under 5 mins)
- Merge to main: full regression suite
- Nightly: full E2E including edge cases and performance checks
- Tag tests by priority: @smoke @regression @critical
- Use Playwright's grep flag to filter by tag in different pipeline stages

What to add: Mention that fast feedback on PRs is more valuable than complete coverage on every commit.

Q6 A Playwright test is failing in CI but the failure is intermittent — sometimes it passes, sometimes it fails. What do you do?

Real context: The most common and frustrating problem in every QA engineer's life.

What a strong answer looks like:

Don't just add retries and call it done. A senior engineer finds the root cause:

- First, tag it as flaky and track the failure rate — don't ignore it
- Use Playwright's --repeat-each flag to reproduce consistently
- Check for race conditions: is the test waiting for something that isn't ready?
- Check for test pollution: is a previous test leaving dirty state?
- Use waitForResponse or waitForLoadState instead of arbitrary timeouts
- If truly intermittent after investigation, quarantine it — don't block CI

What to add: Retries mask problems. Quarantine is acceptable temporarily. Root cause fix is always the goal.

API + UI COMBINED

Q7 How do you use Playwright's API testing alongside UI testing in the same framework?

Real context: Startups need lean frameworks. Separate tools for API and UI is expensive.

What a strong answer looks like:

Playwright's APIRequestContext is underused. Strong candidates know it well:

- Use request.get/post/put/delete for API-only tests — no browser needed
- Use API calls in beforeEach to set up test data instead of UI flows
- Use API calls in afterEach to clean up — faster than UI teardown
- Mix both: API to create user, UI to verify the experience
- Share auth context between API and UI tests using the same storageState

What to add: This approach reduces test execution time by 40-60% in real projects.

Q8 How do you test a feature end-to-end when it depends on a third-party service you can't control in tests?

Real context: Payment gateways, email services, SMS — every startup has external dependencies.

What a strong answer looks like:

Mocking strategy is key. Strong candidates show they've actually done this:

- Use Playwright's route() to intercept and mock external API calls
- Return predictable mock responses — both success and failure scenarios
- Test the happy path with a mock success response
- Test error handling with mock failure/timeout responses
- Keep real integration tests separate — run them less frequently

What to add: Mention that mocking makes tests deterministic. Non-deterministic tests are not reliable tests.

STARTUP STRATEGY

Q9 A startup asks you to get to 70% test coverage in 60 days as the first QA hire. What is your exact approach?

Real context: This is the founding QA challenge. How you answer shows if you've actually done it.

What a strong answer looks like:

Don't start with 70%. Start with what matters most. A strong answer shows a phased approach:

- Week 1-2: Audit the product. Map all user journeys. Identify the top 10 critical paths.
- Week 2-3: Write smoke tests for critical paths. Get CI running. Unblock the team.
- Week 3-5: Expand to regression suite. Cover happy paths for all major features.
- Week 5-8: Add edge cases, negative scenarios, API tests. Hit the 70% target.
- Throughout: Document everything. Involve devs. Make quality a team sport.

What to add: 70% coverage in 60 days is aggressive. Show you understand trade-offs and can communicate risk.

Q10 How do you convince a team that is moving fast that slowing down for testing is worth it?

Real context: Every startup QA engineer faces this. Your answer shows maturity.

What a strong answer looks like:

Don't argue for slowing down. Argue for building quality into speed:

- Show data: how long did the last production bug take to fix vs prevent?
- Frame tests as confidence, not gatekeeping: 'this lets you ship faster'
- Start small: one CI check that catches real bugs builds trust fast
- Make QA invisible in the flow — automated checks nobody has to trigger manually
- Celebrate wins loudly: 'our automated tests caught this before it hit users'

What to add: The best QA engineers don't fight the culture. They change it by demonstrating value quietly and consistently.